SOFTWARE ENGINEERING LAB 6 / COVERAGE ANALYSIS

Code and Branch Coverage

Tennis scoring application | 3 November 2026

Atharva Anand | PES1UG24CS092

Repository and maintained test files

github.com/officialAtharvaAnand/se_lab1_pes1ug24cs092/tree/main/Lab6

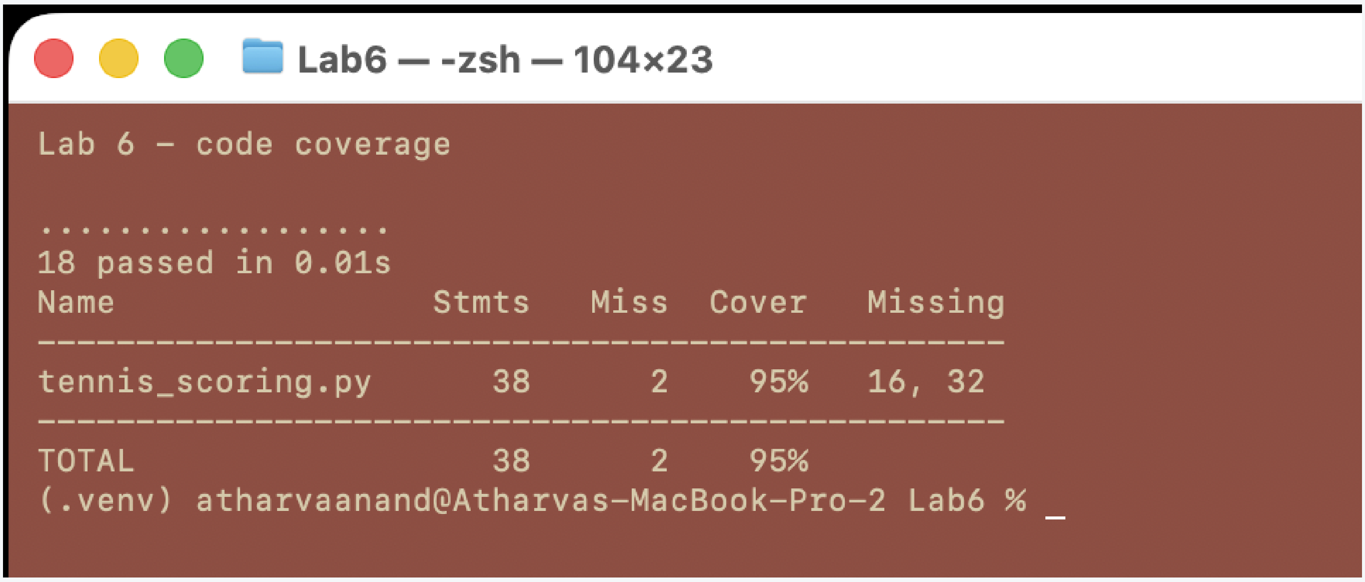
The assigned **tennis_scoring.py** application is unchanged. Two separate Python test files are committed: CodeCoverageTest_tennis_scoring_application.py for line coverage and BranchCoverageTest_tennis_scoring_application.py for branch coverage. Their complete source is reproduced on pages 3-4.

Measured results

Test stage	Line coverage	Branch coverage
Supplied starter (2 tests)	37%	32%
Code coverage (18 tests)	95%	94%
Branch coverage (27 tests)	**100%**	**100%**

Code coverage stage

The first maintained suite passes all 18 cases and executes 36 of 38 statements. The terminal report identifies lines 16 and 32 as still untested.



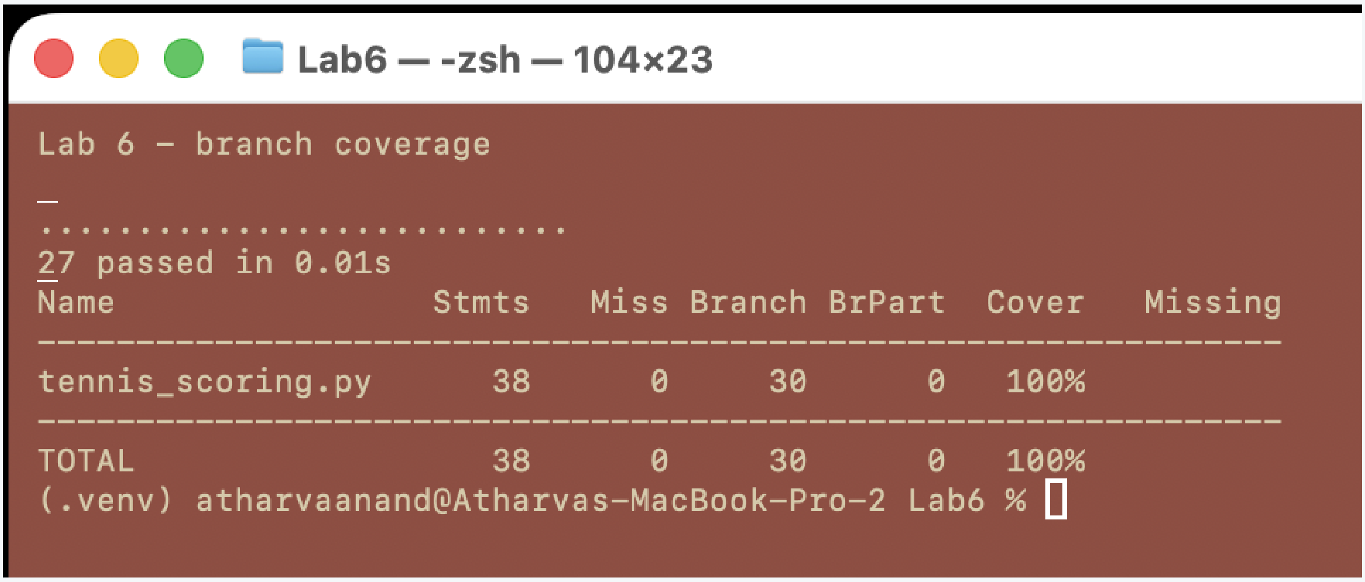
```
Lab 6 - code coverage

..................
18 passed in 0.01s
Name                Stmts   Miss  Cover   Missing
-------------------------------------------------
tennis_scoring.py      38      2    95%   16, 32
-------------------------------------------------
TOTAL                  38      2    95%
(.venv) atharvaanand@Atharvas-MacBook-Pro-2 Lab6 % _
```

Terminal screenshot: 95% line coverage (18 tests passed). Open original screenshot

Branch coverage and interpretation

Final branch coverage

The branch suite passes all 27 cases. Coverage reports 38 of 38 statements and **30 of 30 branches** executed, with no missing or partial branches.



```
Lab 6 - branch coverage
_
...........................
27 passed in 0.01s
Name                Stmts   Miss Branch BrPart  Cover   Missing
---------------------------------------------------------------
tennis_scoring.py      38      0     30      0   100%
---------------------------------------------------------------
TOTAL                  38      0     30      0   100%
(.venv) atharvaanand@Atharvas-MacBook-Pro-2 Lab6 % 
```

Terminal screenshot: 100% branch coverage (27 tests passed).

Open original screenshot

What the additional tests exercise

Game: both invalid-point inputs; ordinary scores; deuce; each advantage; both win outcomes; and one-point versus two-point margins.
Set: undecided, standard two-game wins, 7-5 wins, and both 7-6 tie-break results.
Match: in-progress, A and B wins, and deciding-set states in best-of-three and best-of-five matches.

Interpretation

Line coverage asks whether a statement ran; branch coverage also asks whether each measured decision outcome ran. The code stage reached 95% line coverage but still missed the **advantage B** and **6-7 set winner B** outputs. The final suite covers those paths and the decision boundaries. Coverage is an execution measure, not a proof that the scoring implementation is correct for every possible input.

Verification and commit record

All discoverable tests: **47 passed**. Code stage commit: **9d94116**. Branch stage commit: **9b3c67a**. Both commits are on the repository's main branch.

Appendix A | Code coverage tests

CodeCoverageTest_tennis_scoring_application.py

Committed Python test file used for the 95% line-coverage stage (18 passing cases).

```python
"""Tests for the first, line-coverage stage of Lab 6."""

import pytest

from tennis_scoring import game_score, match_status, set_winner


@pytest.mark.parametrize(
    ("points_a", "points_b", "expected"),
    [
        (0, 0, "0-0"),
        (2, 1, "30-15"),
        (3, 3, "deuce"),
        (5, 5, "deuce"),
        (5, 3, "game A"),
        (3, 5, "game B"),
        (4, 3, "advantage A"),
        (4, 0, "game A"),
        (0, 4, "game B"),
    ],
)
def test_game_scores(points_a, points_b, expected):
    assert game_score(points_a, points_b) == expected


def test_negative_points_are_rejected():
    with pytest.raises(ValueError, match="negative points"):
        game_score(-1, 0)


@pytest.mark.parametrize(
    ("games_a", "games_b", "expected"),
    [
        (3, 2, None),
        (6, 4, "A"),
        (4, 6, "B"),
        (7, 6, "A"),
    ],
)
def test_set_results(games_a, games_b, expected):
    assert set_winner(games_a, games_b) == expected


@pytest.mark.parametrize(
    ("sets_a", "sets_b", "best_of", "expected"),
    [
        (0, 0, 3, "in progress"),
        (2, 0, 3, "A wins"),
        (0, 2, 3, "B wins"),
        (1, 1, 3, "decider"),
    ],
)
def test_match_results(sets_a, sets_b, best_of, expected):
    assert match_status(sets_a, sets_b, best_of) == expected
```

SOFTWARE ENGINEERING LAB 6

Appendix B | Branch coverage tests

BranchCoverageTest_tennis_scoring_application.py

Committed Python test file used for the 100% branch-coverage stage (27 passing cases).

```python
"""Decision and boundary tests for the branch-coverage stage of Lab 6."""

import pytest

from tennis_scoring import game_score, match_status, set_winner


@pytest.mark.parametrize("points_a,points_b", [(-1, 0), (0, -1)])
def test_either_negative_point_is_rejected(points_a, points_b):
    with pytest.raises(ValueError, match="negative points"):
        game_score(points_a, points_b)


@pytest.mark.parametrize(
    ("points_a", "points_b", "expected"),
    [
        (0, 0, "0-0"),
        (2, 3, "30-40"),
        (3, 3, "deuce"),
        (4, 4, "deuce"),
        (5, 3, "game A"),
        (3, 5, "game B"),
        (4, 3, "advantage A"),
        (3, 4, "advantage B"),
        (4, 2, "game A"),
        (2, 4, "game B"),
    ],
)
def test_game_decision_outcomes(points_a, points_b, expected):
    assert game_score(points_a, points_b) == expected


@pytest.mark.parametrize(
    ("games_a", "games_b", "expected"),
    [
        (5, 5, None),
        (6, 5, None),
        (6, 4, "A"),
        (4, 6, "B"),
        (7, 5, "A"),
        (5, 7, "B"),
        (7, 6, "A"),
        (6, 7, "B"),
    ],
)
def test_set_decision_outcomes(games_a, games_b, expected):
    assert set_winner(games_a, games_b) == expected


@pytest.mark.parametrize(
    ("sets_a", "sets_b", "best_of", "expected"),
    [
        (0, 0, 3, "in progress"),
        (2, 0, 3, "A wins"),
        (0, 2, 3, "B wins"),
        (1, 1, 3, "decider"),
        (2, 2, 5, "decider"),
        (3, 1, 5, "A wins"),
        (1, 3, 5, "B wins"),
    ],
)
def test_match_decision_outcomes(sets_a, sets_b, best_of, expected):
    assert match_status(sets_a, sets_b, best_of) == expected
```